Simple. Fast. Human.

Tina4 Ruby – Quick Reference

🔥 **Hot Tips**

- Routes go in `routes/` , templates in `templates/` , static files in `public/`
- GET routes are public by default; POST/PUT/PATCH/DELETE require a token (use `auth: false` to make public)
- Return a `Hash` or `Array` from a route block and it auto-detects as JSON
- Run `tina4 start` to launch the dev server on port 7145
- Auto-generate a full GraphQL API from your ORM classes with `schema.from_orm(User)`

Installation • Static Websites • Routing • Middleware • Templates • Sessions • SCSS • Environments • Authentication • Forms & Tokens • AJAX • OpenAPI • Databases • Database Results • Migrations • ORM • CRUD • REST Client • Testing • Websockets • Queues • GraphQL • WSDL • Localization

Installation

Menu Return to top

```
tina4 init my-project
cd my-project
bundle install
tina4 start
```

More details around project setup and customizations.

Static Websites

Put `.twig` files in `./templates` • assets in `./public`

twig

```twig
<!-- templates/index.twig -->
<h1>Hello Static World</h1>
```

More details on static website routing.

Basic Routing

ruby

```ruby
require "tina4"

Tina4.get "/" do |request, response|
  response.html "<h1>Hello Tina4 Ruby</h1>"
end

# POST requires Bearer auth by default
Tina4.post "/api" do |request, response|
  response.json({ data: request.params })
end

# Redirect after post
Tina4.post "/register" do |request, response|
  response.redirect "/welcome"
end
```

Follow the links for basic routing and dynamic routing with variables.

Middleware

ruby

```ruby
Tina4.before "/api" do |request, response|
  # Runs before any route matching /api*
  Tina4::Debug.info("API request: #{request.path}")
end

Tina4.after do |request, response|
```

```ruby
  # Runs after every route
  response.headers["X-Powered-By"] = "Tina4 Ruby"
end
```

Follow the links for more on Middleware Declaration and Pattern Matching.

Template Rendering

Put `.twig` files in `./templates` • assets in `./public`

twig

```twig
<!-- templates/index.twig -->
<h1>Hello {{ name }}</h1>
```

ruby

```ruby
Tina4.get "/" do |request, response|
  response.render("index.twig", { name: "World!" })
end
```

Sessions

The default session handler is `FileHandler`. Override `SESSION_HANDLER` in `.env`.

Handler	Backend	Required gem
`:file` (default)	File system	—
`:redis`	Redis	`redis`
`:mongo`	MongoDB	`mongo`

ruby

```ruby
Tina4.get "/session/set" do |request, response|
  request.session["name"] = "Joe"
  request.session["info"] = { list: ["one", "two", "three"] }
  response.text "Session Set!"
end

Tina4.get "/session/get" do |request, response|
```

```
    name = request.session["name"]
    info = request.session["info"]
    response.json({ name: name, info: info })
  end
```

SCSS Stylesheets

Drop in `./src/scss` → auto-compiled to `./public/css`

```scss
// src/scss/main.scss
$primary: #2c3e50;
body {
  background: $primary;
  color: white;
}
```

More details on CSS and SCSS.

Environments

Default development environment can be found in `.env`

```
PROJECT_NAME="My Project"
VERSION=1.0.0
TINA4_DEBUG_LEVEL=ALL
API_KEY=ABC1234
DATABASE_URL=sqlite3:data.db
```

```ruby
api_key = ENV["API_KEY"] || "ABC1234"
```

Authentication

Pass `Authorization: Bearer <token>` to secured routes. JWT RS256 keys auto-generated in `.keys/` .

ruby

```ruby
# Public route
Tina4.get "/login" do |request, response|
  token = Tina4::Auth.generate_token({ "user_id" => 1 })
  response.json({ token: token })
end

# Secured route (requires Bearer token)
Tina4.secure_get "/profile" do |request, response|
  response.json({ message: "Welcome!" })
end
```

HTML Forms and Tokens

twig

```twig
<form method="POST" action="/register">
    <input name="email">
    <button>Save</button>
</form>
```

More details on posting form data.

AJAX and tina4helper.js

Tina4 ships with a small javascript library to assist with AJAX calls.

More details on available features.

OpenAPI and Swagger UI

Visit `http://localhost:7145/swagger`

ruby

```ruby
Tina4.get "/users", swagger_meta: { description: "Get all users" } do |request,
  response.json({ users: [] })
end
```

Follow the links for more on Configuration, Usage and Metadata.

Databases

```ruby
# db = Tina4::Database.new("<connection_string>")
db = Tina4::Database.new("sqlite3:data.db")
```

Follow the links for more on Available Connections, Core Methods, Usage and Transactions.

Database Results

```ruby
result = db.fetch("SELECT * FROM users", [], limit: 3, skip: 1)

array = result.to_a        # Array of hashes
json = result.to_json       # JSON string
csv = result.to_csv         # CSV string
```

Looking at detailed Usage will improve deeper understanding.

Migrations

```bash
tina4 migrate --create create_users_table
```

```sql
-- migrations/20260313120000_create_users_table.sql
CREATE TABLE users (
    id   INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT
);
```

```bash
tina4 migrate
```

Migrations have rollback support and status tracking.

ORM

ruby
```ruby
class User < Tina4::ORM
  integer_field :id, primary_key: true, auto_increment: true
  string_field  :name
  string_field  :email
end

User.create(name: "Alice", email: "alice@example.com")

user = User.find(1)
user.name = "Alice Wonder"
user.save
```

ORM functionality is extensive — see the Advanced Detail for the full picture.

CRUD

ruby
```ruby
Tina4.get "/users/dashboard" do |request, response|
  users = User.all
  response.render("users/dashboard.twig", { users: users })
end
```

More details on how CRUD works.

Consuming REST APIs

ruby
```ruby
api = Tina4::API.new("https://api.example.com", auth_header: "Bearer xyz")
result = api.get("/users/42")
puts result.body
```

More details are available on POST bodies, authorization headers, and API responses.

Inline Testing

ruby
```ruby
Tina4.describe "Math operations" do
  it "adds numbers" do
```

```
    assert_equal 4, 2 + 2
  end
end
```

Run: `tina4 test`

Websockets

```ruby
Tina4.get "/ws/chat" do |request, response|
  ws = Tina4::Websocket.new(request)
  ws.on_message do |data|
    ws.send("Echo: #{data}")
  end
  ws.start
end
```

Queues

Supports litequeue (default/SQLite), RabbitMQ, and Kafka backends.

```ruby
queue = Tina4::Queue.new(topic: "emails")
Tina4::Producer.new(queue).produce({ to: "alice@example.com", subject: "Welcome

consumer = Tina4::Consumer.new(queue)
consumer.each do |msg|
  puts msg.data
end
```

GraphQL

Zero-dependency GraphQL with ORM auto-schema generation.

```ruby
gql = Tina4::GraphQL.new
gql.schema.from_orm(User)
```

```
gql.register_route("/graphql")
```

POST queries to `/graphql`, or visit it in a browser for the GraphiQL IDE.

graphql

```
{ users(limit: 5) { id name email } }
```

Full details on manual schema definition, mutations, variables, fragments, and programmatic usage.

WSDL

ruby

```
class Calculator < Tina4::WSDL
  service_url "http://localhost:7145/calculator"

  def add(a, b)
    { result: a + b }
  end
end
```

Localization (i18n)

Set `TINA4_LANGUAGE` in `.env` to change framework language.

ruby

```
puts Tina4.t("server_stopped")  # "Server stopped." (en)
```

↑ Back to top

Next page
Setting up a project